

Option A — Build Your Own WhatsApp Business API SaaS

Developer Requirements Document (Direct-to-Meta, no reseller/BSP)

1. Overview

You build directly on **Meta's own WhatsApp Cloud API** — no reseller, no white-label, no middleman company. You deal with Meta directly for API access, and you own 100% of the code.

There are two sub-models for how billing works with Meta directly — pick one to design around (details in Section 4):

- **Model 1 (Tech Provider):** Each client connects their own WhatsApp number and pays Meta directly from their own Meta Business account. You only charge your own software/subscription fee. Achievable from day one, no special approval needed beyond normal app review.
- **Model 2 (Solution Partner):** You hold the billing relationship — clients top up a wallet with you, you pay Meta, you keep the markup (this is the model LeminAI-type products run on). This requires applying to Meta for elevated partner status, which usually comes after you've shown some real usage — not guaranteed on day one.

Recommendation for a solo/small dev team: Build the whole platform assuming Model 2 architecture (wallet, ledger, markup — since that's the real business model), but launch on Model 1 rules first if Meta hasn't approved your Solution Partner application yet. The code barely changes either way — only who ultimately pays Meta.

2. Documents Required

Document	Why
Business registration / GST certificate	Meta Business Verification + Razorpay KYC
PAN card (business + authorized signatory)	Razorpay, Meta
Address proof (must match everywhere)	Meta Business Verification
Authorized signatory's government ID	Meta Business Verification
Bank account details / cancelled cheque	Razorpay settlement
Privacy Policy (published on your live website)	Mandatory for WhatsApp API approval
Refund & Cancellation Policy (published on website)	Mandatory for Razorpay

Critical rule: business name must match **exactly** across Meta account, registration docs, and website footer — this is the #1 reason Meta rejects verification.

3. Meta / WhatsApp API — Technical Setup Requirements

1. **Meta Business Portfolio** — create at business.facebook.com, complete Business Verification.
2. **Meta Developer account** — create at developers.facebook.com, create a new App (type: Business).
3. **WhatsApp product added to your app** — gives you a test WABA + test number to start coding immediately, before verification finishes.
4. **System User + permanent access token** — for server-to-server API calls (not the 24-hour temporary token).
5. **Webhook endpoint (HTTPS, publicly reachable)** — Meta sends all incoming messages and status updates (delivered/read/failed) here. Needs signature verification (`X-Hub-Signature-256`).
6. **Embedded Signup integration** — Meta's official popup flow that lets each of your clients connect their own WABA/number to your app in a few clicks, without you handling their Meta login manually. This is what makes your platform multi-tenant.
7. **App Review** — required to go from test mode to live/production, for permissions like `whatsapp_business_management` and `whatsapp_business_messaging`. Meta reviews your actual app + use case (may ask for a screen recording of your flow).
8. **Message template creation/approval** — templates (marketing/utility/authentication) are created via API (`POST /message_templates`) and must be approved individually before use.
9. **Phone number registration** — each client's number gets registered to their WABA via API; if it was on personal WhatsApp/WhatsApp Business app, it must be deregistered first (or use Coexistence where available).
10. **Sending limits** — new numbers start at a lower daily tier (roughly 1K → 10K → 100K → unlimited), which increases with verification + good quality rating. Build your system to handle/display this — don't assume unlimited sending from day one.
11. **(For Model 2 later) Solution Partner application** — submitted through Meta's partner program once you're ready; grants the credit-line/billing capability across all your clients.

4. Payment System (your own — Razorpay, no reseller involved)

Flow 1 — Subscription billing

- Razorpay Subscriptions API — create Plans, generate checkout, listen for webhooks (`subscription.charged`, `subscription.cancelled`, `payment.failed`) to control access in your DB.

Flow 2 — Wallet/conversation billing (Model 2 above)

- `wallet_ledger` table per client: balance + transaction history.
- Recharge via Razorpay Payment Links/Orders → credited on webhook confirmation.
- Every outbound message: deduct (Meta's base rate + your markup) from the client's wallet **in the same atomic DB transaction** as the send — this must use row-level locking to prevent race conditions during bulk sends.

Flow 1 only (Model 1 above)

- If running Model 1, you skip the message-level deduction — client's Meta costs bill on their own Meta account. You only run Flow 1 (subscription).

5. Software Architecture

Message flow

```
OUTBOUND (campaign/broadcast)
Your Dashboard → Your Backend API → Job Queue (rate-limited per client)
    → Meta Cloud API → WhatsApp → End customer
    |
    Delivery/Read Webhook → Backend → DB → Dashboard/Analytics

INBOUND (customer reply)
End customer → WhatsApp → Meta → Webhook (HTTPS POST) → Your Backend
    → Match tenant + contact by phone_number_id → Save message
    → Chatbot engine checks for automation match
    → No match / handoff requested → Push to agent's live inbox (WebSocket)
```

Core data model

Table	Purpose
<code>tenants</code>	One row per client business/workspace
<code>users / roles</code>	Admin, agent, manager access per tenant
<code>channels</code>	Per-tenant WABA connection — phone_number_id, WABA ID, encrypted access token, webhook verify token
<code>contacts / segments</code>	Audience lists, tags, custom fields
<code>conversations / messages</code>	Chat history, status (sent/delivered/read/failed), direction
<code>templates</code>	Name, category, language, Meta approval status, variable placeholders
<code>campaigns</code>	Segment, template, schedule, per-recipient send log
<code>flows</code>	Chatbot builder — JSON graph of nodes/edges, trigger keywords
<code>wallet_ledger</code>	Balance + itemized transactions (only needed for Model 2)

Table	Purpose
subscriptions	Plan, billing cycle, feature entitlements
webhook_logs	Raw Meta webhook payloads — for debugging/audit

Key backend responsibilities

- **Webhook handler:** verify signature, route by `phone_number_id` to correct tenant, deduplicate (Meta can resend the same webhook).
- **Send queue:** never call Meta's send API synchronously from a user request for bulk campaigns — always push to a queue with per-tenant rate limiting and retry/backoff logic.
- **Template sync:** poll or webhook-listen for template approval status changes from Meta.
- **Flow engine:** on each inbound message, load the contact's current flow state, evaluate the next node, execute (send message / call an API / wait / handoff).

6. Tech Stack

Layer	Recommendation
Backend	Node.js (NestJS) or Python (FastAPI/Django)
Frontend	React/Next.js + Tailwind, React Flow for the chatbot builder canvas
Database	PostgreSQL
Queue/cache	Redis + BullMQ (or RabbitMQ)
Realtime	Socket.io / Pusher / Ably — for live agent inbox
Object storage	AWS S3 / Cloudflare R2 — for media messages
Hosting	AWS/GCP/DigitalOcean, Mumbai region
Payments	Razorpay (Subscriptions + Payment Links APIs)
Messaging	Meta WhatsApp Cloud API directly (<code>graph.facebook.com/v.../messages</code>)

7. Development Phases

1. **Setup (parallel with coding):** Business registration, Meta Business Verification, Razorpay KYC, Meta Developer app creation.
2. **Core messaging (3–4 weeks):** Webhook receiver, send API integration, single-tenant message send/receive working end to end with a test number.
3. **Multi-tenant + Embedded Signup (2–3 weeks):** Let real clients connect their own WABA; tenant isolation in DB.

4. **Inbox + templates (2 weeks):** Shared inbox UI, agent assignment, template creation/approval flow.
5. **Billing (2–3 weeks):** Razorpay subscriptions, wallet ledger, atomic deduction logic.
6. **Campaigns + automation (3–4 weeks):** Broadcast queue, segmentation, chatbot flow builder.
7. **Analytics + polish (2 weeks):** Delivery/read tracking dashboards, quality-rating monitoring.
8. **App Review + go live:** Submit to Meta for production access, test with one real client before public launch.

Realistic total: ~**3–4 months** for a solo/small dev team to MVP, assuming Meta verification runs in parallel and doesn't block coding.

8. Cost (build-it-yourself, no reseller fees)

Item	Cost
Meta Business Verification	Free
Meta Developer account + API access	Free
Hosting (MVP scale)	₹3,000 – ₹15,000/month
Domain + website	₹1,000 – ₹3,000/year
Razorpay	No setup fee, small % per transaction
Your development time	Your own — this is the real "cost," budget 3–4 months

No reseller/platform fees in this model — the only ongoing external cost is hosting, and Meta's own per-message rates (which your clients' wallet payments cover).